MIT | Academy of Engineering

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

PROJECT REPORT

Practical 6 : Retrieval-Augmented Generation for Document Question Answering

Submitted by :

Name : Tanishka Vishnu Tapkir

PRN : 202401110071

Batch : A3

Abstract

This practical builds a document question-answering application using Retrieval-Augmented Generation (RAG). Instead of asking a language model to answer only from its general training, the system first searches a document collection and provides relevant passages as evidence. The implementation cleans and chunks text, creates semantic embeddings, indexes the chunks in a Chroma vector database, retrieves the top-k passages and sends them to a pre-trained LLM. The same questions are also answered without retrieval. The comparison focuses on answer relevance, evidence support, completeness and unsupported claims. The accompanying Streamlit application makes the workflow visible by showing both answers and the retrieved passages.

1. Introduction

Pre-trained Large Language Models (LLMs) can produce fluent answers, but they may not know the details of a document uploaded after their training. When a question asks for a particular value, rule or instruction from a source document, a general answer may be incomplete or incorrect. Retrieval-Augmented Generation helps by finding relevant text first and then using that text while generating the response.

The main idea is: retrieve evidence, then generate an answer. The retriever searches the indexed collection. The generator reads the question together with the retrieved passages. RAG does not guarantee correctness; the selected passages must be relevant and the generated answer should be checked against them.

2. Aim

Aim: To implement a RAG-based document question-answering application using a pre-trained LLM and compare it with the same LLM without retrieval.

Problem statement: A general-purpose LLM may not have access to the exact information in a user-provided document and may give unsupported details. The system should retrieve useful source passages and use them as context, then compare the response against an LLM-only baseline.

3. Objectives

- Load and clean domain-specific documents.
- Split the documents into smaller chunks with suitable overlap.
- Generate semantic embeddings for the chunks and questions.
- Create a vector database, index the chunks and retrieve relevant passages.
- Integrate retrieval with a pre-trained LLM using a context-grounded prompt.
- Generate a baseline response without retrieval using the same LLM.
- Review answer quality, evidence support and limitations.

4. System Design

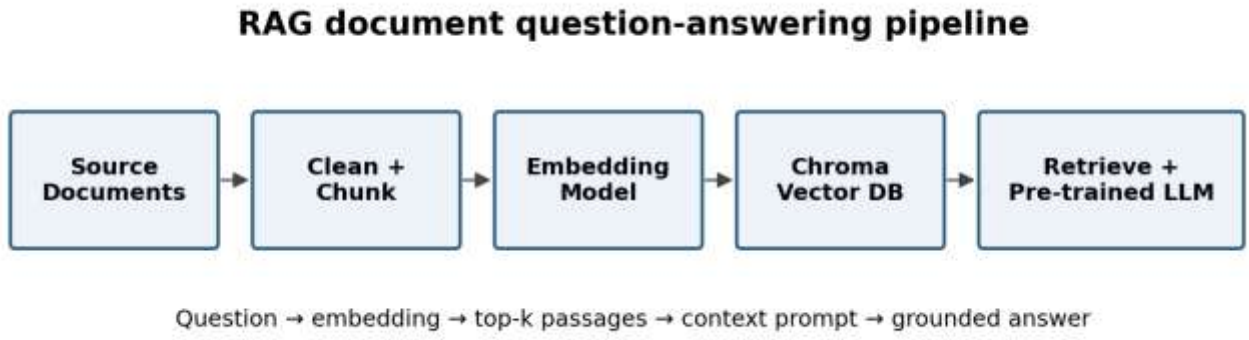


Figure 1. Proposed RAG workflow.

The document-processing stage prepares clean chunks. The embedding model converts each chunk into a vector. Chroma stores the vectors with source metadata. At question time, the query vector is compared with stored vectors and the most relevant passages are returned. The LLM receives the question and the retrieved context to generate the RAG answer.

5. Document Processing, Chunking and Embeddings

5.1 Document collection and cleaning

The demonstration collection contains short knowledge notes on RAG overview, system components, chunking, embeddings, LLM limitations, retrieval quality and evaluation. In a practical application, this collection can be replaced with course notes, manuals or other approved TXT/PDF documents. Cleaning removes repeated whitespace and unwanted characters while preserving the actual meaning of the source.

5.2 Chunking strategy

Long documents are split into smaller passages because retrieving a whole document may include too much unrelated information. The implementation uses a word-based chunk size with a small overlap between adjacent chunks. Overlap helps preserve context when a sentence or key idea crosses a chunk boundary. Chunk size and overlap are parameters that can be adjusted after testing.

Parameter	Implementation choice	Reason
Chunk length	80 words (starter value)	Keeps each passage focused and small.
Overlap	15 words	Preserves some context between neighboring chunks.
Metadata	Source name and chunk index	Helps identify where a retrieved passage came from.

5.3 Embedding generation

6. Vector Database and Retrieval

ChromaDB is used as the vector database. Each record stores the chunk text, its embedding and metadata such as source and chunk index. When a user asks a question, the system creates the query embedding and asks Chroma for the nearest top-k passages. The retrieved text is then assembled into a context block.

Component	Role in the system
Chroma collection	Stores chunk vectors, text and metadata.
Query embedding	Represents the user's question as a vector.
Cosine distance	Ranks passages by vector similarity; it is not an answer-accuracy score.
Top-k retrieval	Returns a selected number of passages for the prompt.

7. Pre-trained LLM and Prompt Design

The generation model used in the implementation is google/flan-t5-base. The RAG prompt provides the retrieved context and asks the model to answer from it. It also instructs the model to say when the context is insufficient. The baseline prompt asks the same model to answer using general knowledge, without retrieved passages. Using the same generator for both conditions makes the comparison more controlled.

8. Application Demonstration

The Streamlit application provides a question field, a top-k selector and a button to compare answers. The page displays the RAG answer and the LLM-only answer side by side. Below them, it lists the retrieved evidence, source labels and Chroma distance values. TXT documents can be uploaded and indexed into the vector collection.

Application feature	Purpose
Question input	Allows the user to ask about the document collection.
TXT upload	Adds user documents to the indexed knowledge base.
Top-k selector	Controls how many passages are retrieved.
Two answer panels	Makes the RAG and no-retrieval responses easy to compare.
Evidence display	Shows the passages that were provided to the RAG prompt.

To run the application: install the libraries listed in requirements_rag.txt and execute `streamlit run rag_app.py` in the project folder.

8.1 Application Screenshots

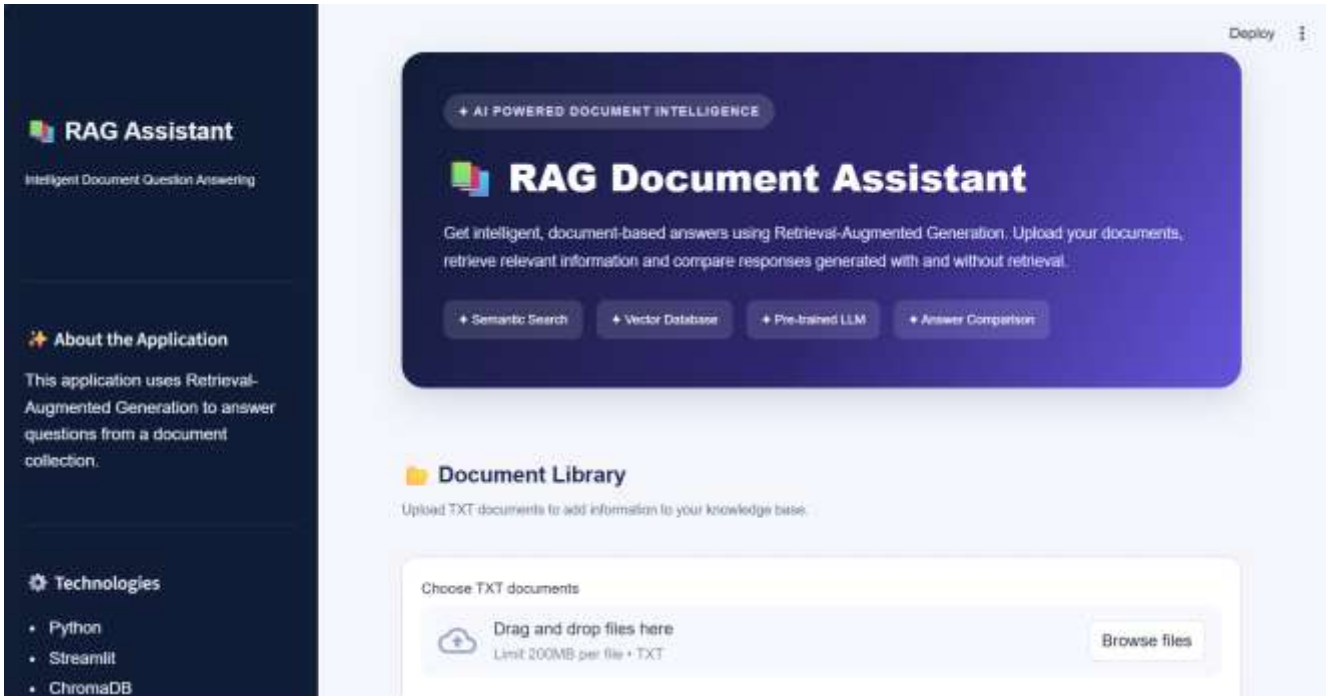


Figure 2: Home screen of the RAG Document Assistant showing the application dashboard and navigation panel.

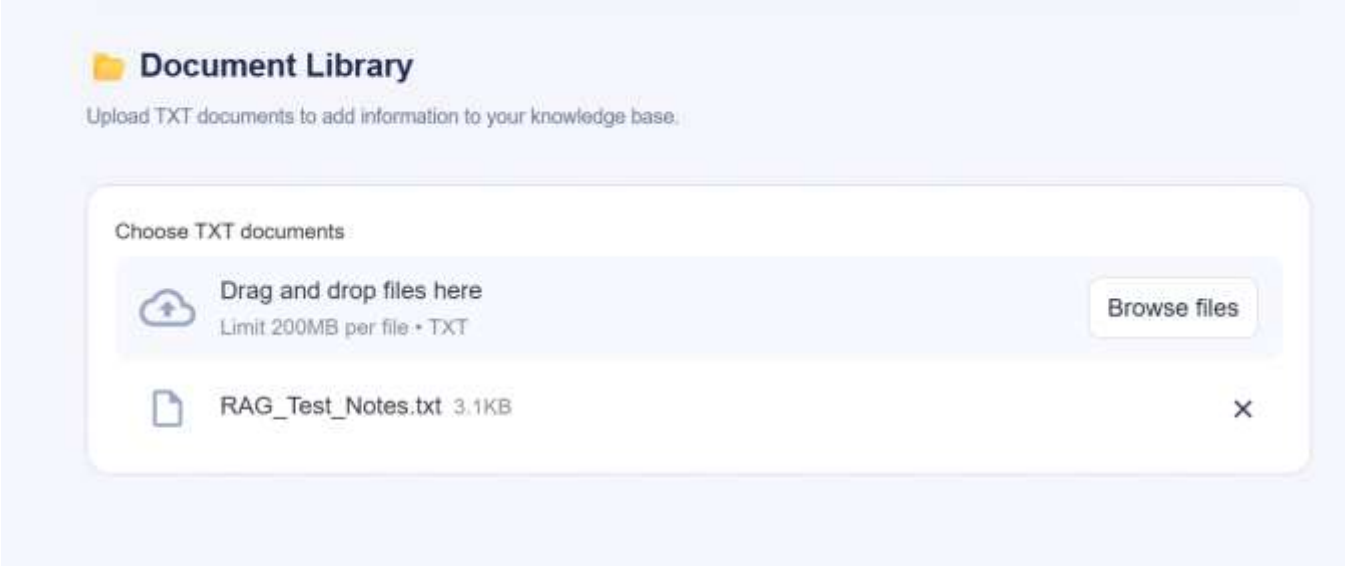


Figure 3: Document upload interface used to add TXT files to the knowledge base.

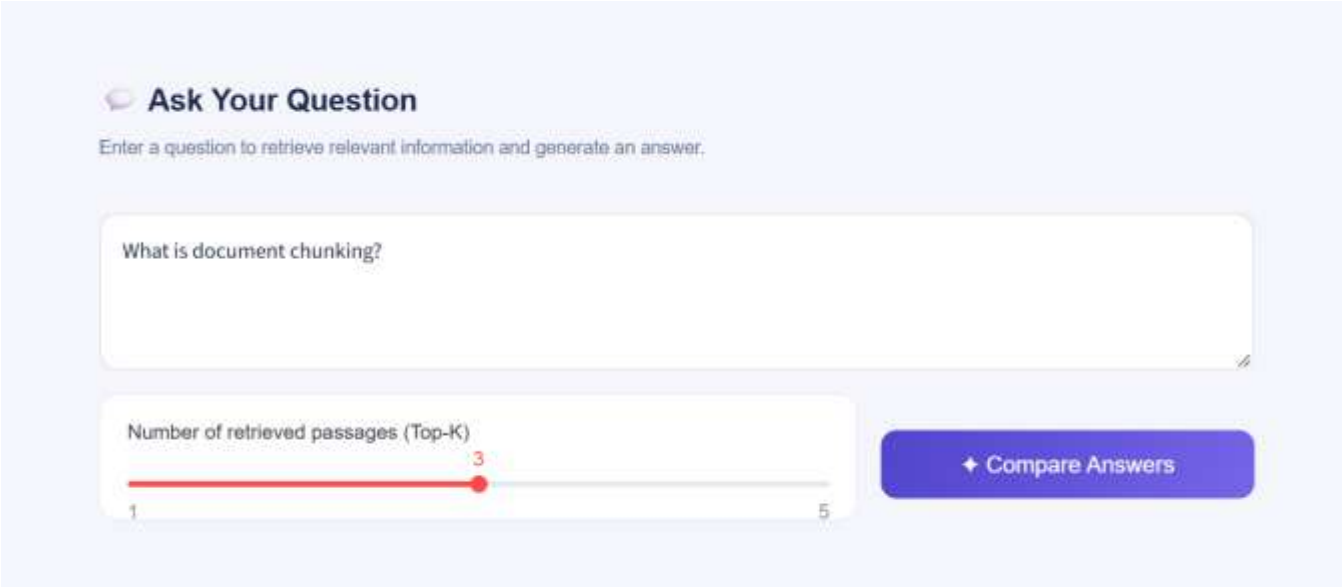


Figure 4: User entering a question for document-based question answering.

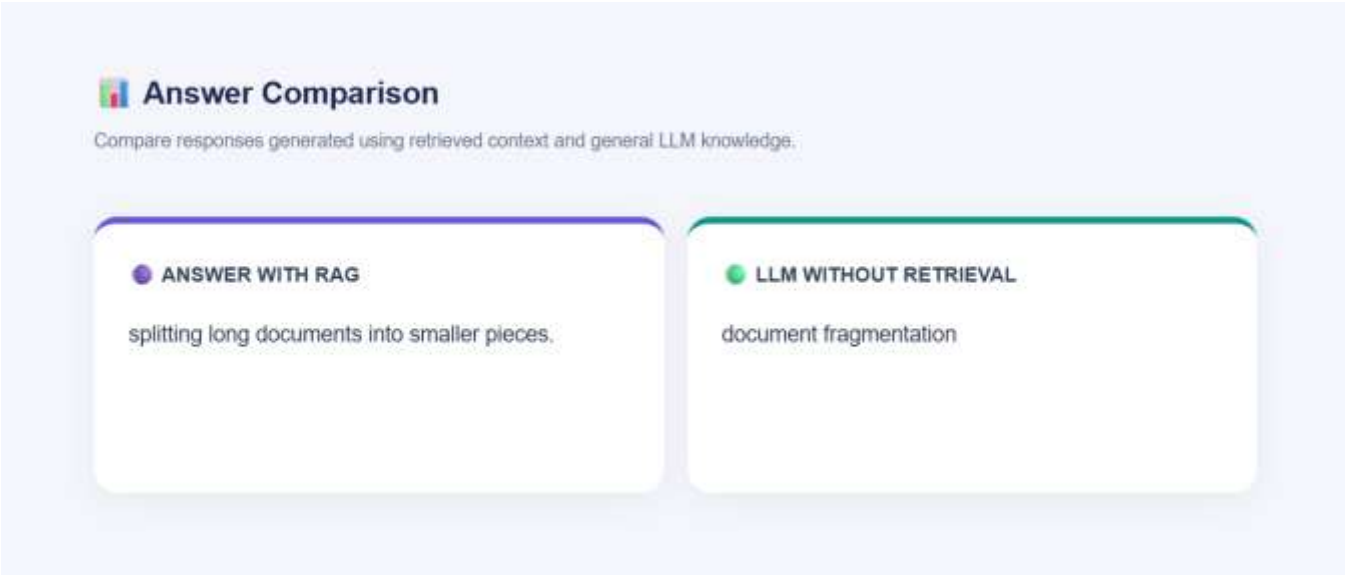


Figure 5: Comparison of answers generated with RAG and without retrieval using the pre-trained LLM.



Figure 6: Retrieved document passages displayed with their source names and Chroma similarity distances.

9. Experimental Setup

The comparison uses the same question set in two conditions: (1) RAG, where the generator receives retrieved passages, and (2) LLM-only, where the generator receives only the question. For each question, the retrieved evidence and both answers should be saved and reviewed.

Condition	Generator input	What it helps examine
With RAG	Question + top-k retrieved passages	Whether evidence improves document-specific answers.
Without retrieval	Question only	What the model can answer from general learned knowledge.

10. Evaluation Criteria

Metric	Evaluation method
Relevance	Does the response answer the user's question?
Evidence support	Are the main claims supported by the retrieved source text?
Completeness	Are the important facts needed for the question included?
Unsupported claims	Does the answer include details not supported by the source?

Retrieval quality	Are the retrieved passages useful for answering the question?

A simple manual scale can be used: 1 = weak, 2 = partly correct, 3 = strong. Scores should be assigned only after inspecting the actual output and comparing it with the source passages.

11. Results and Comparison

The actual model answers and scores must be collected by running the notebook or application. The table below is included for recording observed results. It is intentionally left blank rather than presenting made-up model performance.

Test question	RAG relevance (1–3)	RAG evidence support (1–3)	LLM-only relevance (1–3)	Observation
What are the main parts of RAG?	1	1	1	RAG returned only the document title instead of explaining the two components. LLM-only generated an unrelated answer.
Why are documents split into chunks?	2	3	2	RAG correctly mentioned context preservation through overlap, but the explanation was incomplete. LLM-only gave a partially relevant answer.
What does an embedding represent?	2	3	1	RAG correctly identified text meaning but did not explain its numerical vector representation. LLM-only gave an unclear answer.
How does the vector database help?	1	3	1	RAG avoided generating unsupported information because the retrieved passages did not explain Chroma. LLM-only generated an unrelated answer.
What can happen with irrelevant retrieval?	3	3	1	RAG correctly explained that unrelated passages can result in weak or incorrect answers. LLM-only generated an unrelated answer.

11.1 Discussion

RAG is expected to be more useful when a question depends on information in the source collection and the retriever selects the correct passages. The LLM-only answer may still be useful for general questions, but it cannot directly inspect the supplied documents. RAG can also fail: irrelevant passages, missing information or a model misunderstanding the context can lead to weak answers. The final discussion should be updated based on the actual test outputs.

12. Limitations

- The model may still misread the context or produce unsupported statements.
- Retrieval quality depends on the document text, chunk size, embedding model and top-k setting.
- The sample knowledge base is small and is only meant to demonstrate the workflow.
- Model downloads require internet access and enough memory; CPU execution may be slow.
- Manual evaluation can involve judgement, so clear scoring rules are important.

13. Conclusion

This practical implements a complete RAG workflow using document processing, chunking, embeddings, a Chroma vector database, similarity-based retrieval and a pre-trained LLM. It also provides an LLM-only baseline for comparison. The application displays the retrieved evidence so users can inspect what was used to generate an answer. The main learning is that RAG can improve document-grounded responses, but it still requires relevant retrieval and careful evaluation.

14. References

- Chroma documentation: https://docs.trychroma.com/
- Sentence-Transformers documentation: https://www.sbert.net/
- Hugging Face Transformers documentation: https://huggingface.co/docs/transformers
- Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” 2020.